

UNIVERSITAT POLITÈCNICA DE CATALUNYA

Escola Tècnica Superior d'Enginyeria de Telecomunicacions de Barcelona (ETSETB)

Practice 2

Grau en Enginyeria Física

Authors: Arnau Torret & Arnau Montull

Subject: SIMCOM

Date: October 23, 2026

1 Exercise 1

1.1 Question

Simulation of a simple Lennard-Jones fluid, such as liquid argon, with a hand-made FORTRAN MD code which incorporates the Leap-frog Verlet algorithm.

STEP 1: Understanding the code.

- Open the FORTRAN file `leapfroglj.f` and have a look.
- Note that you'll need 2 auxiliary files: (1) `leap-lj.data` and (2) `leap-conf.data`, containing (1) data input and (2) initial coordinates and velocities, respectively.
- The main program is ready to simulate a LJ liquid of 500 atoms of mass 39.948 uma, $\sigma = 3.405$ Å and $\varepsilon/k_B = 119.8$ K (Argon), with a time step of $\Delta t = 0.01$ ps. Remember that the LJ potential is of the form:

$$V_{\mathrm{LJ}}(r) = 4\varepsilon\left[\left(\frac{\sigma}{r}\right)^{12} - \left(\frac{\sigma}{r}\right)^{6}\right]. \tag{1}$$

- Compile the code and make a run of 10000 time steps. If you compute the temperature T and total energy E (kinetic + potential) you will get the following figures.

1.2 Code

This step runs the leap-frog program supplied with the practice, transcribed to free-form Fortran without algorithmic changes; only the accumulation and printing of the averages was added. The run uses the parameters of the statement (10000 steps, $N = 500$). The full program, with every later addition, is listed in the Annex.

```
   sumt = 0.d0
   sumt2 = 0.d0
   sume = 0.d0
   sume2 = 0.d0

   ! 5. Start the loop to generate new configurations
   do i = 1, nconf
      call forces(natoms, r, boxlength, accel, rc, epot)
      call velpos(natoms, vinf, accel, deltat, r, nf, ecin, temp, &
                  boxlength)
      etot = ecin + epot
      write(3,*) i*deltat, etot
      write(4,*) i*deltat, temp
      sumt = sumt + temp
      sumt2 = sumt2 + temp*temp
      sume = sume + etot
      sume2 = sume2 + etot*etot
   end do
   close(3)
   close(4)

   ! Mean and standard deviation of T and E_tot over the run
   tmean = sumt/nconf
   emean = sume/nconf
   tsd = dsqrt(sumt2/nconf - tmean**2)
   esd = dsqrt(sume2/nconf - emean**2)
   print *, 'steps  = ', nconf
   print *, '<T>    = ', tmean, '  sigma_T = ', tsd
   print *, '<Etot> = ', emean, '  sigma_E = ', esd
```

Listing 1: Main loop with the running sums of T and E_{tot} (added) and the printed means and standard deviations.

1.3 Results

With $L^* = 8.427$ ($\rho^* = 0.835$) and $\Delta t^* = 4.637 \times 10^{-3}$ ($\tau = 2.156$ ps), the 10000 steps cover $t^* = 46.37$ (100 ps). Figure 1 shows T and $E_{\mathrm{tot}} = E_{\mathrm{kin}} + E_{\mathrm{pot}}$.

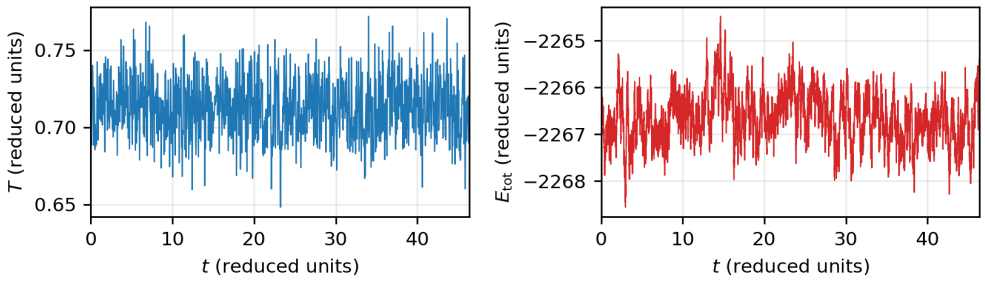


Figure 1: Temperature $T = 2E_{\mathrm{kin}}/(3N - 3)$ (left) and total truncated energy E_{tot} (right), in reduced units.

1.4 Discussion & Notes

- **Energy:** leap-frog Verlet is symplectic, so E_{tot} should only oscillate. We get $\langle E_{\mathrm{tot}} \rangle = -2266.65$, and $\sigma_E = 0.537$ (0.024%), a very small fluctuation.
- **Temperature:** $\langle T^* \rangle = 0.713$ (85.4 K), $\sigma_T = 0.0175$ (2.5%). That is a much larger relative fluctuation than for E_{tot}, as E_{kin} and E_{pot} are exchanged. Both curves match the reference figures for a dense liquid.

2 Exercise 2

2.1 Question

STEP 2: Analysing the structure and calculating thermodynamic properties.

- Modify the code to calculate the radial distribution function (RDF) $g(r)$ from the MD simulation. Compute the RDF for a run of 10000 steps. Plot $g(r)$ vs. r using reduced units.
- Modify the code to calculate the pressure P. Compute the mean values of the total and different contributions to E and P. Include the tail corrections. Analyze the convergence of the results as a function of the simulation length. Use reduced units.

2.2 Code

Only the lines added for the radial distribution function, the virial pressure and the long-range tails are shown. The full program is listed in the Annex.

```
9     double precision mass, nid
10
11    ! 1. Defining dimensions
12    dimension r(3,1000), vinf(3,1000), accel(3,1000)
13    parameter (nhis = 1000)
14    dimension g(nhis)
```

Listing 2: Histogram storage. `nid` is real because a name starting with `n` would be an integer under implicit typing.

```
49    pi = 4.d0*datan(1.d0)
50    delg = boxlength/(2.d0*dfloat(nhis))
51    rho = dfloat(natoms)/boxlength**3
52    etail = (8.d0*pi/3.d0)*rho*dfloat(natoms)* &
53            (1.d0/(3.d0*rc**9) - 1.d0/rc**3)
54    ptail = (16.d0*pi/3.d0)*rho**2* &
55            (2.d0/(3.d0*rc**9) - 1.d0/rc**3)
56    do j = 1, nhis
57       g(j) = 0.d0
58    end do
59    ekin_sum = 0.d0
60    epot_sum = 0.d0
61    pkin_sum = 0.d0
62    pvir_sum = 0.d0
```

Listing 3: Tail corrections and zeroing of the RDF and pressure accumulators before the loop.

```
74        ! Conserved NVE energy (no tail). Thermodynamic totals include it.
75        etot = ecin + epot
76        etot_corr = etot + etail
77        pkin = 2.d0*ecin/(3.d0*boxlength**3)
78        pvir = vir/(3.d0*boxlength**3)
79        ptot = pkin + pvir + ptail
80        ekin_sum = ekin_sum + ecin
81        epot_sum = epot_sum + epot
82        pkin_sum = pkin_sum + pkin
83        pvir_sum = pvir_sum + pvir
84        write(3,*) i*deltat, etot
85        write(4,*) i*deltat, temp
86        write(8,*) i*deltat, etot_corr, pkin, pvir, ptail, ptot
```

Listing 4: Energy and pressure contributions accumulated on every step of the MD loop.

```
92    ekin_avg = ekin_sum/dfloat(nconf)
93    epot_avg = epot_sum/dfloat(nconf)
94    etot_avg = ekin_avg + epot_avg + etail
95    pkin_avg = pkin_sum/dfloat(nconf)
96    pvir_avg = pvir_sum/dfloat(nconf)
97    ptot_avg = pkin_avg + pvir_avg + ptail
```

Listing 5: Run averages of the energy and pressure contributions.

```
120   ! 6. Radial distribution function in reduced units.
121   open(15, file='g-leap.dat', status='unknown')
122   do j = 1, nhis
123      rr = delg*(dfloat(j) - 0.5d0)
124      vb = (dfloat(j)**3 - dfloat(j-1)**3)*delg**3
125      nid = (4.d0/3.d0)*pi*vb*rho
126      g(j) = g(j)/(dfloat(nconf)*dfloat(natoms)*nid)
127      write(15,*) rr, g(j)
128   end do
129   close(15)
```

Listing 6: Normalisation of $g(r)$ by the ideal-gas content of each shell.

```
183   epot = 0.d0
184   vir = 0.d0
185
186   ! atom-atom interactions
187   do is = 1, natoms-1
188      do js = is+1, natoms
189         call lj(is, js, r, boxlength, accel, rc, pot, virij, nhis, g, delg)
190         epot = epot + pot
191         vir = vir + virij
192      end do
193   end do
```

Listing 7: Pair virial summed inside `forces`.

```
230      virij = forcedist*rr2
231   end if
232
233   ! Histogram of pair distances up to L/2 (minimum image).
234   if (rr .lt. boxlength/2.d0) then
235      ig = int(rr/delg) + 1
236      if (ig .ge. 1 .and. ig .le. nhis) g(ig) = g(ig) + 2.d0
237   end if
```

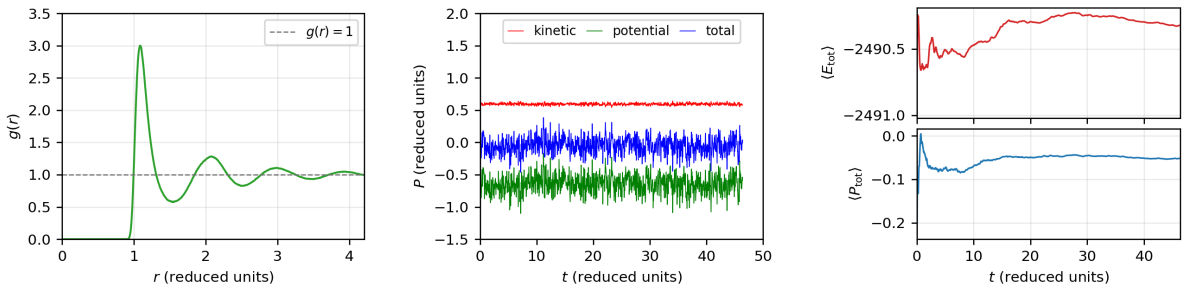
Listing 8: End of subroutine `lj`: virial of one pair inside the cutoff, and the $g(r)$ shell counted up to $L/2$.

2.3 Results

The same 10000-step trajectory as in Exercise 1 ($N = 500$, $\rho^* = 0.835$, $\langle T^* \rangle = 0.713$, $r_c = 2.5$) was used. $g(r)$ is accumulated in 1000 shells up to $L^*/2 = 4.21$ and normalised by the ideal-gas content of each shell. The pressure is $P = P_{\text{kin}} + P_{\text{vir}} + P_{\text{tail}}$ with $P_{\text{kin}} = 2E_{\text{kin}}/(3V)$ and $P_{\text{vir}} = \frac{1}{3V}\sum_{i<j} \vec{r}_{ij} \cdot \vec{f}_{ij}$. The tails assume $g(r) = 1$ beyond r_c:

$$E_{\text{tail}} = \frac{8\pi}{3}\rho N\left(\frac{1}{3r_c^9} - \frac{1}{r_c^3}\right) = -223.67, \qquad P_{\text{tail}} = \frac{16\pi}{3}\rho^2\left(\frac{2}{3r_c^9} - \frac{1}{r_c^3}\right) = -0.7465. \tag{2}$$

Figure 2 shows $g(r)$, the pressure contributions and the running means of the totals. Table 1 gives the mean of every contribution after increasing simulation lengths. The last row is the full run.



(a) RDF (b) Pressure (c) Convergence

Figure 2: Structure and thermodynamics of the Lennard-Jones liquid in reduced units. (a) Radial distribution function $g(r)$. (b) Kinetic, potential ($P_{\mathrm{vir}} + P_{\mathrm{tail}}$) and total pressure versus time. (c) Running means of E_{tot} and P_{tot} (tails included) from $t = 0$ to t.

Table 1: Mean energy and pressure contributions (reduced units) after n steps. $E_{\mathrm{tail}} = -223.67$ and $P_{\mathrm{tail}} = -0.7465$ are constant and are included in the totals.

n	t^*	$\langle E_{\mathrm{kin}} \rangle$	$\langle E_{\mathrm{pot}} \rangle$	$\langle E_{\mathrm{tot}} \rangle$	$\langle P_{\mathrm{kin}} \rangle$	$\langle P_{\mathrm{vir}} \rangle$	$\langle P_{\mathrm{tot}} \rangle$
1000	4.64	535.49	−2802.35	−2490.53	0.5965	0.0721	−0.0779
2500	11.59	534.82	−2801.60	−2490.45	0.5958	0.0922	−0.0585
5000	23.19	534.06	−2800.66	−2490.27	0.5949	0.1042	−0.0473
7500	34.78	533.35	−2799.94	−2490.26	0.5941	0.1061	−0.0462
10000	46.37	533.66	−2800.31	−2490.32	0.5945	0.1001	−0.0519

2.4 Discussion & Notes

- **RDF:** $g(r) = 0$ inside the repulsive core ($r \lesssim 0.93$) and then oscillates as in a dense liquid. The first peak is at $r = 1.089$ ($g = 3.00$), the first minimum at $r = 1.544$ ($g = 0.576$), and further peaks at $r = 2.079$ ($g = 1.28$) and $r = 2.985$ ($g = 1.11$). The oscillations die out and $g(L/2) = 0.993 \approx 1$: there is short-range order but no long-range order.

- **Energy:** $\langle E_{\mathrm{tot}} \rangle / N = -4.98$ per particle, and the tail is 9% of E_{tot}.

- **Pressure:** $\langle P_{\mathrm{kin}} \rangle = 0.5945 \approx \rho^* \langle T^* \rangle = 0.596$. The repulsive virial 0.100 and the attractive tail -0.746 almost cancel it, so $P_{\mathrm{tot}} = -0.052$ is close to zero. Without the tail P^* would be $+0.69$, so the tail correction is essential at $r_c = 2.5$.

- **Convergence:** $\langle E_{\mathrm{tot}} \rangle$ changes by only 0.2 (less than 0.01%) once a few thousand steps have been averaged. $\langle P_{\mathrm{tot}} \rangle$ converges more slowly: it is a small difference of large terms. It settles after about 5000 steps ($t^* \approx 23$) to within ± 0.006.

3 Exercise 3

3.1 Question

STEP 3: Analysing the dynamics.

- Modify the code to calculate the velocity autocorrelation function (VACF) and the mean square displacement (MSD):

$$C(t) = \langle \vec{v}(t) \cdot \vec{v}(0) \rangle, \tag{3}$$

$$r^2(t) = \langle [\vec{r}(t) - \vec{r}(0)]^2 \rangle. \tag{4}$$

  Compute the VACF and the MSD for a run of 10000 steps. Plot the results.

- Write a code to calculate the spectral density (SDW) of the VACF:

$$S(\omega) = \int_0^\infty dt\, C(t) \cos(\omega t). \tag{5}$$

  Use a Simpson 3/8 rule or similar. Plot the results.

- Use Einstein's diffusion formula to obtain the self-diffusion coefficient D:

$$D = \lim_{t\to\infty} \frac{1}{6t} r^2(t). \tag{6}$$

3.2 Code

Only the lines added in this step are shown, with a few lines of the existing program kept where they mark the insertion point. The full program is listed in the Annex.

```
11    ! 1. Defining dimensions
12    dimension r(3,1000), vinf(3,1000), accel(3,1000), runw(3,1000), rold(3,1000)
13    dimension c(0:10000), r2(0:10000)
14    parameter (nhis = 1000)
15    dimension g(nhis)
16    dimension t(0:10000), D(10000), c_spectrum(0:1000)
17    ! Multiple time origins for VACF and MSD (lecture SAMPLE algorithm).
18    ! init=0 initialize, init=1 accumulate a sample, init=2 normalize.
19    integer :: ncor, it0, ntmax, t0max, ntel, t0
20    integer, allocatable :: time0(:), ntime(:)
21    double precision, allocatable :: vx0(:,:,:), x0(:,:,:), vacf(:), r2t(:)
```

Listing 9: Unwrapped-trajectory arrays `runw`, `rold`, and storage for the VACF, the MSD, the spectral density and the time-origin buffers.

```
51    ! 4. Change to reduced units
52    call reduced(natoms, r, vinf, boxlength, deltat, epsil, sigma, &
53                 mass, uvel)
54
55    ! runw holds positions without periodic wrapping, needed for the MSD.
56    runw = r
```

Listing 10: Unwrapped coordinates, initialised once the positions are in reduced units.

```
78    ! Correlation window of about 5 reduced time units: the VACF has
79    ! decayed and the MSD is already linear. ncor is a multiple of 6 so
80    ! the Simpson 3/8 rule has a complete set of panels. A new time
81    ! origin is taken every it0 samples; t0max keeps every origin still
82    ! inside the window. ntmax is tmax in the lecture listing (1-based,
83    ! with C(0) stored in vacf(1)).
84    it0 = 5
```

```
    ncor = nint(5.d0/deltat)
    ncor = ncor - mod(ncor, 6)
    if (ncor < 6) ncor = 6
    if (ncor > nconf) ncor = nconf - mod(nconf, 6)
    ntmax = ncor + 2
    t0max = ncor/it0 + 3
    allocate(time0(t0max), ntime(ntmax))
    allocate(vx0(3, natoms, t0max), x0(3, natoms, t0max))
    allocate(vacf(ntmax), r2t(ntmax))
    call sample(0)
```

Listing 11: Correlation window and circular buffer of time origins.

```
       call forces(natoms, r, boxlength, accel, rc, epot, vir, nhis, g, delg)
       rold = r
       call velpos(natoms, vinf, accel, deltat, r, nf, ecin, temp, boxlength)

       ! Unfold this step (SAMPLE does not apply periodic wrapping to r2t).
       do j = 1, natoms
          do l = 1, 3
             dr = r(l,j) - rold(l,j)
             runw(l,j) = runw(l,j) + dr - boxlength*dnint(dr/boxlength)
          end do
       end do
       call sample(1)
```

Listing 12: Each leap-frog step is unfolded before `sample` accumulates $C(t)$ and $r^2(t)$.

```
    ! 7. Normalize VACF and MSD (SAMPLE init=2) and copy to 0-based arrays.
    call sample(2)
    do i = 0, ncor
       c(i) = vacf(i+1)
       r2(i) = r2t(i+1)
       t(i) = i*deltat
    end do
    print *, 'VACF and MSD averaged over time origins'
    print *, '  origins at t = 0     =', ntime(1)
    print *, '  origins at t = tmax  =', ntime(ncor+1)
    print *, '  C(0) =', c(0)

    open(16, file='c-leap.dat', status='unknown')
    open(17, file='r2-leap.dat', status='unknown')
    do i = 0, ncor
       write(16,*) t(i), c(i)
       write(17,*) t(i), r2(i)
    end do
    close(16)
    close(17)

    ! 8. Spectral density of the averaged VACF, Simpson 3/8 rule.
    !    Frequencies w = 0, 1, ..., 100. Only those are written: the
    !    previous loop stopped at 100 and the rest of the file was unset.
    call spectrum(c(0:ncor), t(0:ncor), c_spectrum)
    open(18, file='c_spectrum-leap.dat', status='unknown')
    do i = 0, 100
       write(18,*) dfloat(i), c_spectrum(i)
    end do
    close(18)
```

Listing 13: Normalisation and output of the VACF and the MSD, and spectral density of $C(t)$.

```
    ! 10. Least-squares linear fit r2(t) = 6*D*t + b over the diffusive regime.
    !     The first quarter of the window (ballistic regime) is discarded.
```

```
!     yint is used instead of "intercept": implicit typing would make it integer.
ifit = ncor/4
npts = ncor - ifit + 1
sx = 0.d0
sy = 0.d0
sxx = 0.d0
sxy = 0.d0
do i = ifit, ncor
   sx = sx + t(i)
   sy = sy + r2(i)
   sxx = sxx + t(i)*t(i)
   sxy = sxy + t(i)*r2(i)
end do
slope = (dfloat(npts)*sxy - sx*sy)/(dfloat(npts)*sxx - sx*sx)
yint = (sy - slope*sx)/dfloat(npts)
d_einstein = slope/6.d0
tmin = t(ifit)
tmax = t(ncor)
```

Listing 14: Einstein self-diffusion coefficient from a linear fit of $r^2(t)$ in the diffusive regime.

```
!*************************************************************
!     SAMPLE: VACF and MSD with multiple time origins
!     init=0 initialize counters
!     init=1 accumulate a sample of data
!     init=2 average the results
!     Positions in x0 / runw are unwrapped (PBC not assumed).
!*************************************************************

subroutine sample(init)
  implicit none
  integer, intent(in) :: init
  integer :: i, j, l, tt0, torg, delt
  double precision :: dx

  if (init == 0) then
     ntel = 0
     t0 = 0
     do i = 1, ntmax
        ntime(i) = 0
        vacf(i) = 0.d0
        r2t(i) = 0.d0
     end do
  else if (init == 1) then
     ntel = ntel + 1
     if (mod(ntel, it0) == 0) then
        t0 = t0 + 1
        tt0 = mod(t0-1, t0max) + 1
        time0(tt0) = ntel
        do j = 1, natoms
           do l = 1, 3
              x0(l,j,tt0) = runw(l,j)
              vx0(l,j,tt0) = vinf(l,j)
           end do
        end do
     end if
     do torg = 1, min(t0, t0max)
        delt = ntel - time0(torg) + 1
        if (delt < ntmax) then
           ntime(delt) = ntime(delt) + 1
           do j = 1, natoms
              do l = 1, 3
                 vacf(delt) = vacf(delt) + vinf(l,j)*vx0(l,j,torg)
                 dx = runw(l,j) - x0(l,j,torg)
```

```
                  r2t(delt) = r2t(delt) + dx*dx
               end do
            end do
         end if
      end do
   else if (init == 2) then
      do i = 1, ntmax
         if (ntime(i) > 0) then
            vacf(i) = vacf(i)/(dfloat(natoms)*dfloat(ntime(i)))
            r2t(i) = r2t(i)/(dfloat(natoms)*dfloat(ntime(i)))
         end if
      end do
   end if
end subroutine sample
```

Listing 15: Internal subroutine `sample`: VACF and MSD with multiple time origins.

```
!*************************************************************
!     spectral density, composite Simpson 3/8 rule
!     S(w) = integral C(t) cos(w t) dt, w = 0, 1, ..., 100
!*************************************************************

subroutine spectrum(C, t, result)
  implicit none
  double precision, dimension(:), intent(in) :: C, t
  double precision, dimension(0:), intent(out) :: result
  integer :: i, w, n, panel
  double precision :: h, s
  double precision, allocatable :: f(:)

  n = int(0.25*size(t))
  h = t(2) - t(1)
  allocate(f(n))
  result = 0.d0

  do w = 0, 100
     do i = 1, n
        f(i) = C(i)*cos(dble(w)*t(i))
     end do

     ! Composite Simpson 3/8: panels come in groups of three.
     s = f(1) + f(n)
     do i = 2, n - 1
        if (mod(i - 1, 3) == 0) then
           s = s + 2.d0*f(i)
        else
           s = s + 3.d0*f(i)
        end if
     end do
     result(w) = 3.d0*h/8.d0*s
  end do

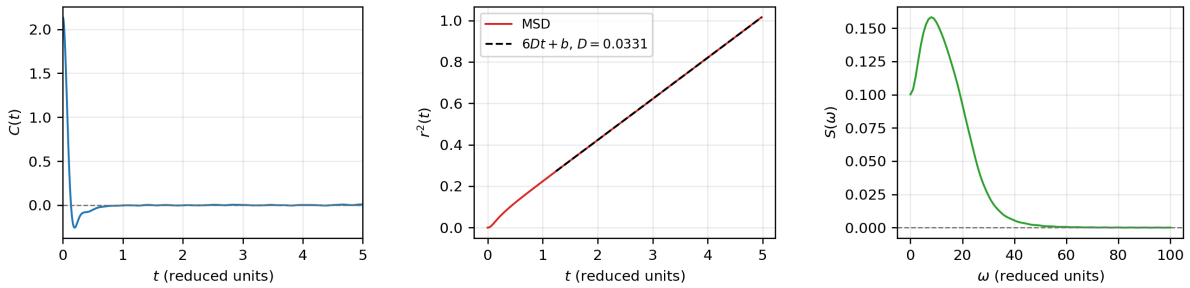
  deallocate(f)
end subroutine spectrum
```

Listing 16: Internal subroutine `spectrum`: composite Simpson 3/8 cosine transform of the VACF.

3.3 Results

On the same 10000-step trajectory ($N = 500$, $\rho^* = 0.835$, $\langle T^* \rangle = 0.713$), $C(t)$ and $r^2(t)$ were averaged over particles and time origins in a window of 1074 steps ($t^* \leq 4.98$). Figure 3 shows

the VACF, the MSD with its Einstein fit, and the spectral density $S(\omega)$. The fit gives a slope of 0.1988 and an intercept of 0.025, so $D^* = 0.0331$.



(a) VACF (b) MSD (c) SDW

Figure 3: Dynamics of the Lennard-Jones liquid in reduced units. (a) Velocity autocorrelation $C(t)$. (b) Mean square displacement $r^2(t)$; the dashed line is the fit $r^2 = 6Dt + b$ on $t^* \in [1.24, 4.98]$. (c) Spectral density $S(\omega)$ of the VACF, Simpson 3/8 rule, $\omega = 0, 1, \ldots, 100$.

3.4 Discussion & Notes

- **Method and setup:** Following the lecture SAMPLE scheme, a new time origin is stored every 5 steps in a circular buffer. The MSD uses unwrapped positions: each leap-frog displacement is unfolded with the minimum image, so $r^2(t)$ is not folded back by the periodic box. The window $n_{\text{cor}} = 1074$ is a multiple of 6, so Simpson's 3/8 rule has complete panels, and it is long enough for $C(t)$ to vanish and for $r^2(t)$ to become linear. The integral (5) is evaluated on $t^* \leq 1.24$ (first quarter of the window), beyond which $C(t) \approx 0$. Following (6), D is one sixth of the slope of $r^2(t)$ fitted over the last three quarters of the window, which discards the ballistic rise.

- **Results:**

  - *VACF:* $C(0) = 2.138$ agrees with the equipartition value $\langle v^2 \rangle = 3\langle T^* \rangle = 2.139$ ($m = 1$). $C(t)$ decays quickly, crosses zero at $t^* = 0.139$ and reaches a minimum $C = -0.256$ at $t^* = 0.195$. Beyond $t^* \approx 0.64$, $|C| < 0.02$ and $C(t)$ only fluctuates around zero.
  - *MSD:* $r^2(t)$ follows the ballistic law $\langle v^2 \rangle t^2$ only for $t^* \lesssim 0.05$ ($r^2 = 0.0053$ against 0.0056), and is linear for $t^* \gtrsim 1$. It reaches $r^2 = 1.016$ at $t^* = 4.98$.
  - *SDW:* $S(\omega)$ peaks at $\omega = 8$ ($S = 0.158$). It falls to 5.4×10^{-3} at $\omega = 40$ and 5×10^{-5} at $\omega = 100$. $S(0) = 0.100$ is finite and $S(0)/3 = 0.0334$ matches the Einstein D^* to 1%.

Annex

The program listed here is the full source used throughout the practice. It is the leap-frog integrator supplied for Exercise 1, together with the radial distribution function, the virial pressure and the tail corrections added in Exercise 2, and the velocity autocorrelation, mean square displacement, spectral density and self-diffusion coefficient added in Exercise 3. Each exercise section quotes only the lines introduced in that step.

```
!***************************************************************************
!          Molecular Dynamics code to simulate at NVE ensemble
!          a system of N atoms of Ar inside a cubic box, interacting
!          through a truncated Lennard-Jones force field at 2.5 sigma.
!          Leap-frog Verlet integration algorithm.
!***************************************************************************
program leapfroglj
   implicit double precision(a-h,o-z)
   double precision mass, nid

   ! 1. Defining dimensions
   dimension r(3,1000), vinf(3,1000), accel(3,1000), runw(3,1000), rold(3,1000)
   dimension c(0:10000), r2(0:10000)
   parameter (nhis = 1000)
   dimension g(nhis)
   dimension t(0:10000), D(10000), c_spectrum(0:1000)
   ! Multiple time origins for VACF and MSD (lecture SAMPLE algorithm).
   ! init=0 initialize, init=1 accumulate a sample, init=2 normalize.
   integer :: ncor, it0, ntmax, t0max, ntel, t0
   integer, allocatable :: time0(:), ntime(:)
   double precision, allocatable :: vx0(:,:,:), x0(:,:,:), vacf(:), r2t(:)

   INCLUDE '../../chdir_to_code.inc'

   ! 2. Reading data and computing related quantities
   open(1, file='../data/leap-lj.data', status='old')
   read(1,*) nconf
   read(1,*) natoms
   read(1,*) mass
   read(1,*) sigma, epsil
   read(1,*) deltat
   close(1)

   nf = 3*natoms - 3  ! number of degrees of freedom
   rc = 2.5d0         ! cutoff radius in units of sigma

   ! 3. Reading initial configuration (positions, velocities) in A and A/ps
   open(2, file='../data/leap-conf.data', status='old')
   do is = 1, natoms
      read(2,*) (r(l,is), l=1,3)
      read(2,*) (vinf(l,is), l=1,3)
   end do
   read(2,*) boxlength
   close(2)

   ! Opening files to write results
   open(3, file='energy-leap.dat', status='unknown')
   open(4, file='temp-leap.dat', status='unknown')
   open(8, file='thermo-leap.dat', status='unknown')

   ! 4. Change to reduced units
   call reduced(natoms, r, vinf, boxlength, deltat, epsil, sigma, &
                mass, uvel)

   ! runw holds positions without periodic wrapping, needed for the MSD.
   runw = r

```

```
58      ! 5. Start the loop to generate new configurations
59      pi = 4.d0*datan(1.d0)
60      delg = boxlength/(2.d0*dfloat(nhis))
61      rho = dfloat(natoms)/boxlength**3
62      etail = (8.d0*pi/3.d0)*rho*dfloat(natoms)* &
63              (1.d0/(3.d0*rc**9) - 1.d0/rc**3)
64      ptail = (16.d0*pi/3.d0)*rho**2* &
65              (2.d0/(3.d0*rc**9) - 1.d0/rc**3)
66      do j = 1, nhis
67         g(j) = 0.d0
68      end do
69      ekin_sum = 0.d0
70      epot_sum = 0.d0
71      pkin_sum = 0.d0
72      pvir_sum = 0.d0
73      sumt = 0.d0
74      sumt2 = 0.d0
75      sume = 0.d0
76      sume2 = 0.d0
77
78      ! Correlation window of about 5 reduced time units: the VACF has
79      ! decayed and the MSD is already linear. ncor is a multiple of 6 so
80      ! the Simpson 3/8 rule has a complete set of panels. A new time
81      ! origin is taken every it0 samples; t0max keeps every origin still
82      ! inside the window. ntmax is tmax in the lecture listing (1-based,
83      ! with C(0) stored in vacf(1)).
84      it0 = 5
85      ncor = nint(5.d0/deltat)
86      ncor = ncor - mod(ncor, 6)
87      if (ncor < 6) ncor = 6
88      if (ncor > nconf) ncor = nconf - mod(nconf, 6)
89      ntmax = ncor + 2
90      t0max = ncor/it0 + 3
91      allocate(time0(t0max), ntime(ntmax))
92      allocate(vx0(3, natoms, t0max), x0(3, natoms, t0max))
93      allocate(vacf(ntmax), r2t(ntmax))
94      call sample(0)
95
96      write(8,'(a)') '# t etot pkin pvir ptail ptot'
97
98      print *, 'Iniciando simulacion MD (', nconf, ' pasos)...'
99      do i = 1, nconf
100        if (mod(i, 1000) == 0) then
101           print *, '  -> Completado paso ', i, ' de ', nconf
102        end if
103        call forces(natoms, r, boxlength, accel, rc, epot, vir, nhis, g, delg)
104        rold = r
105        call velpos(natoms, vinf, accel, deltat, r, nf, ecin, temp, boxlength)
106
107        ! Unfold this step (SAMPLE does not apply periodic wrapping to r2t).
108        do j = 1, natoms
109           do l = 1, 3
110              dr = r(l,j) - rold(l,j)
111              runw(l,j) = runw(l,j) + dr - boxlength*dnint(dr/boxlength)
112           end do
113        end do
114        call sample(1)
115
116        ! Conserved NVE energy (no tail). Thermodynamic totals include it.
117        etot = ecin + epot
118        etot_corr = etot + etail
119        pkin = 2.d0*ecin/(3.d0*boxlength**3)
120        pvir = vir/(3.d0*boxlength**3)
```

```
121         ptot = pkin + pvir + ptail
122         ekin_sum = ekin_sum + ecin
123         epot_sum = epot_sum + epot
124         pkin_sum = pkin_sum + pkin
125         pvir_sum = pvir_sum + pvir
126         write(3,*) i*deltat, etot
127         write(4,*) i*deltat, temp
128         write(8,*) i*deltat, etot_corr, pkin, pvir, ptail, ptot
129         sumt = sumt + temp
130         sumt2 = sumt2 + temp*temp
131         sume = sume + etot
132         sume2 = sume2 + etot*etot
133      end do
134      close(3)
135      close(4)
136      close(8)
137
138      ! Mean and standard deviation of T and E_tot over the run
139      tmean = sumt/nconf
140      emean = sume/nconf
141      tsd = dsqrt(sumt2/nconf - tmean**2)
142      esd = dsqrt(sume2/nconf - emean**2)
143      print *, 'steps  = ', nconf
144      print *, '<T>    = ', tmean, '  sigma_T = ', tsd
145      print *, '<Etot> = ', emean, '  sigma_E = ', esd
146
147      ekin_avg = ekin_sum/dfloat(nconf)
148      epot_avg = epot_sum/dfloat(nconf)
149      etot_avg = ekin_avg + epot_avg + etail
150      pkin_avg = pkin_sum/dfloat(nconf)
151      pvir_avg = pvir_sum/dfloat(nconf)
152      ptot_avg = pkin_avg + pvir_avg + ptail
153
154      open(9, file='averages-leap.dat', status='unknown')
155      write(9,*) 'nconf', nconf
156      write(9,*) 'ekin', ekin_avg
157      write(9,*) 'epot', epot_avg
158      write(9,*) 'etail', etail
159      write(9,*) 'etot', etot_avg
160      write(9,*) 'pkin', pkin_avg
161      write(9,*) 'pvir', pvir_avg
162      write(9,*) 'ptail', ptail
163      write(9,*) 'ptot', ptot_avg
164      close(9)
165      print *, 'Mean values in reduced units'
166      print *, '  Ekin  =', ekin_avg
167      print *, '  Epot  =', epot_avg
168      print *, '  Etail =', etail
169      print *, '  Etot  =', etot_avg
170      print *, '  Pkin  =', pkin_avg
171      print *, '  Pvir  =', pvir_avg
172      print *, '  Ptail =', ptail
173      print *, '  Ptot  =', ptot_avg
174
175      ! 6. Radial distribution function in reduced units.
176      open(15, file='g-leap.dat', status='unknown')
177      do j = 1, nhis
178         rr = delg*(dfloat(j) - 0.5d0)
179         vb = (dfloat(j)**3 - dfloat(j-1)**3)*delg**3
180         nid = (4.d0/3.d0)*pi*vb*rho
181         g(j) = g(j)/(dfloat(nconf)*dfloat(natoms)*nid)
182         write(15,*) rr, g(j)
183      end do
```

```
184     close(15)
185
186     ! 7. Normalize VACF and MSD (SAMPLE init=2) and copy to 0-based arrays.
187     call sample(2)
188     do i = 0, ncor
189        c(i) = vacf(i+1)
190        r2(i) = r2t(i+1)
191        t(i) = i*deltat
192     end do
193     print *, 'VACF and MSD averaged over time origins'
194     print *, '  origins at t = 0     =', ntime(1)
195     print *, '  origins at t = tmax  =', ntime(ncor+1)
196     print *, '  C(0) =', c(0)
197
198     open(16, file='c-leap.dat', status='unknown')
199     open(17, file='r2-leap.dat', status='unknown')
200     do i = 0, ncor
201        write(16,*) t(i), c(i)
202        write(17,*) t(i), r2(i)
203     end do
204     close(16)
205     close(17)
206
207     ! 8. Spectral density of the averaged VACF, Simpson 3/8 rule.
208     !    Frequencies w = 0, 1, ..., 100. Only those are written: the
209     !    previous loop stopped at 100 and the rest of the file was unset.
210     call spectrum(c(0:ncor), t(0:ncor), c_spectrum)
211     open(18, file='c_spectrum-leap.dat', status='unknown')
212     do i = 0, 100
213        write(18,*) dfloat(i), c_spectrum(i)
214     end do
215     close(18)
216
217     ! 9. Instantaneous diffusion coefficient from the mean square displacement
218     open(19, file='D-leap.dat', status='unknown')
219     do i = 1, ncor
220        D(i) = r2(i)/(6.d0*t(i))
221        write(19,*) t(i), D(i)
222     end do
223     close(19)
224
225     ! 10. Least-squares linear fit r2(t) = 6*D*t + b over the diffusive regime.
226     !     The first quarter of the window (ballistic regime) is discarded.
227     !     yint is used instead of "intercept": implicit typing would make it integer.
228     ifit = ncor/4
229     npts = ncor - ifit + 1
230     sx = 0.d0
231     sy = 0.d0
232     sxx = 0.d0
233     sxy = 0.d0
234     do i = ifit, ncor
235        sx = sx + t(i)
236        sy = sy + r2(i)
237        sxx = sxx + t(i)*t(i)
238        sxy = sxy + t(i)*r2(i)
239     end do
240     slope = (dfloat(npts)*sxy - sx*sy)/(dfloat(npts)*sxx - sx*sx)
241     yint = (sy - slope*sx)/dfloat(npts)
242     d_einstein = slope/6.d0
243     tmin = t(ifit)
244     tmax = t(ncor)
245
246     ! Green-Kubo: D = (1/3) * integral of C(t) from 0 to the correlation window.
```

```
247     d_gk = deltat*(sum(c(0:ncor)) - 0.5d0*c(0) - 0.5d0*c(ncor))/3.d0
248
249     open(20, file='diffusion-leap.dat', status='unknown')
250     write(20,'(a)') '# D_einstein slope intercept tmin tmax D_greenkubo'
251     write(20,*) d_einstein, slope, yint, tmin, tmax, d_gk
252     close(20)
253     print *, 'Diffusion coefficient (reduced units)'
254     print *, '  D (Einstein fit) =', d_einstein
255     print *, '  D (Green-Kubo)   =', d_gk
256
257     ! 7. Saving last configuration in A and A/ps
258     open(11, file='newconf.data', status='unknown')
259     do is = 1, natoms
260        write(11,*) (r(l,is)*sigma, l=1,3)
261        write(11,*) (vinf(l,is)*uvel, l=1,3)
262     end do
263     write(11,*) boxlength*sigma
264     close(11)
265
266     stop
267
268   contains
269
270   !*************************************************************
271   !     SAMPLE: VACF and MSD with multiple time origins
272   !     init=0 initialize counters
273   !     init=1 accumulate a sample of data
274   !     init=2 average the results
275   !     Positions in x0 / runw are unwrapped (PBC not assumed).
276   !*************************************************************
277
278   subroutine sample(init)
279     implicit none
280     integer, intent(in) :: init
281     integer :: i, j, l, tt0, torg, delt
282     double precision :: dx
283
284     if (init == 0) then
285        ntel = 0
286        t0 = 0
287        do i = 1, ntmax
288           ntime(i) = 0
289           vacf(i) = 0.d0
290           r2t(i) = 0.d0
291        end do
292     else if (init == 1) then
293        ntel = ntel + 1
294        if (mod(ntel, it0) == 0) then
295           t0 = t0 + 1
296           tt0 = mod(t0-1, t0max) + 1
297           time0(tt0) = ntel
298           do j = 1, natoms
299              do l = 1, 3
300                 x0(l,j,tt0) = runw(l,j)
301                 vx0(l,j,tt0) = vinf(l,j)
302              end do
303           end do
304        end if
305        do torg = 1, min(t0, t0max)
306           delt = ntel - time0(torg) + 1
307           if (delt < ntmax) then
308              ntime(delt) = ntime(delt) + 1
309              do j = 1, natoms
```

```
               do l = 1, 3
                  vacf(delt) = vacf(delt) + vinf(l,j)*vx0(l,j,torg)
                  dx = runw(l,j) - x0(l,j,torg)
                  r2t(delt) = r2t(delt) + dx*dx
               end do
            end do
         end if
      end do
   else if (init == 2) then
      do i = 1, ntmax
         if (ntime(i) > 0) then
            vacf(i) = vacf(i)/(dfloat(natoms)*dfloat(ntime(i)))
            r2t(i) = r2t(i)/(dfloat(natoms)*dfloat(ntime(i)))
         end if
      end do
   end if
end subroutine sample

!*********************************************************
!     spectral density, composite Simpson 3/8 rule
!     S(w) = integral C(t) cos(w t) dt, w = 0, 1, ..., 100
!*********************************************************

subroutine spectrum(C, t, result)
  implicit none
  double precision, dimension(:), intent(in) :: C, t
  double precision, dimension(0:), intent(out) :: result
  integer :: i, w, n, panel
  double precision :: h, s
  double precision, allocatable :: f(:)

  n = int(0.25*size(t))
  h = t(2) - t(1)
  allocate(f(n))
  result = 0.d0

  do w = 0, 100
     do i = 1, n
        f(i) = C(i)*cos(dble(w)*t(i))
     end do

     ! Composite Simpson 3/8: panels come in groups of three.
     s = f(1) + f(n)
     do i = 2, n - 1
        if (mod(i - 1, 3) == 0) then
           s = s + 2.d0*f(i)
        else
           s = s + 3.d0*f(i)
        end if
     end do
     result(w) = 3.d0*h/8.d0*s
  end do

  deallocate(f)
end subroutine spectrum

end program leapfroglj

!*********************************************************
!              subroutine reduced
!*********************************************************

subroutine reduced(natoms, r, vinf, boxlength, deltat, &
```

```
                      epsil, sigma, mass, uvel)
  implicit double precision(a-h,o-z)
  double precision mass
  dimension r(3,1000), vinf(3,1000)

  rgas = 8.314472673d0  ! J/(mol*K)
  utime = sigma*dsqrt(mass/epsil)*dsqrt(10.d0/rgas)
  uvel = sigma/utime    ! unit of velocity, expressed in A/ps

  boxlength = boxlength/sigma
  deltat = deltat/utime
  do is = 1, natoms
     do l = 1, 3
        r(l,is) = r(l,is)/sigma
        vinf(l,is) = vinf(l,is)/uvel
     end do
  end do

  return
end subroutine reduced

!*********************************************************
!              subroutine forces
!*********************************************************

subroutine forces(natoms, r, boxlength, accel, rc, epot, vir, nhis, g, delg)
  implicit double precision(a-h,o-z)
  dimension r(3,1000), accel(3,1000)
  dimension g(nhis)

  do is = 1, natoms
     do l = 1, 3
        accel(l,is) = 0.d0  ! sets accelerations to 0
     end do
  end do
  epot = 0.d0
  vir = 0.d0

  ! atom-atom interactions
  do is = 1, natoms-1
     do js = is+1, natoms
        call lj(is, js, r, boxlength, accel, rc, pot, virij, nhis, g, delg)
        epot = epot + pot
        vir = vir + virij
     end do
  end do

  return
end subroutine forces

!*********************************************************
!              subroutine Lennard-Jones
!*********************************************************

subroutine lj(is, js, r, boxlength, accel, rc, pot, virij, nhis, g, delg)
  implicit double precision(a-h,o-z)
  dimension r(3,1000), accel(3,1000)
  dimension rij(3)
  dimension g(nhis)

  rr2 = 0.d0
  pot = 0.d0
  virij = 0.d0
```

```
436
437      do l = 1, 3
438         rijl = r(l,js) - r(l,is)
439         rij(l) = rijl - boxlength*dnint(rijl/boxlength)
440         rr2 = rr2 + rij(l)*rij(l)
441      end do
442
443      rr = dsqrt(rr2)
444
445      if (rr .lt. rc) then
446         ynvrr2 = 1.d0/rr2
447         ynvrr6 = ynvrr2*ynvrr2*ynvrr2
448         ynvrr12 = ynvrr6*ynvrr6
449         forcedist = 24.d0*(2.d0*ynvrr12 - ynvrr6)*ynvrr2
450         pot = 4.d0*(ynvrr12 - ynvrr6)
451         do l = 1, 3
452            accel(l,is) = accel(l,is) - forcedist*rij(l)
453            accel(l,js) = accel(l,js) + forcedist*rij(l)
454         end do
455         virij = forcedist*rr2
456      end if
457
458      ! Histogram of pair distances up to L/2 (minimum image).
459      if (rr .lt. boxlength/2.d0) then
460         ig = int(rr/delg) + 1
461         if (ig .ge. 1 .and. ig .le. nhis) g(ig) = g(ig) + 2.d0
462      end if
463
464      return
465    end subroutine lj
466
467    !*************************************************************
468    !              subroutine velpos
469    !*************************************************************
470
471    subroutine velpos(natoms, vinf, accel, deltat, r, nf, ecin, temp, &
472                      boxlength)
473      implicit double precision(a-h,o-z)
474      dimension vinf(3,1000), accel(3,1000), r(3,1000)
475
476      ecin = 0.d0
477      do is = 1, natoms
478         v2 = 0.d0
479         do l = 1, 3
480            vsup = vinf(l,is) + accel(l,is)*deltat
481            r(l,is) = r(l,is) + vsup*deltat
482            v = (vsup + vinf(l,is))/2.d0
483            v2 = v2 + v*v
484            vinf(l,is) = vsup
485         end do
486         ecin = ecin + 0.5d0*v2
487      end do
488      temp = 2.d0*ecin/dfloat(nf)
489
490      ! Applying periodic boundary conditions
491      do is = 1, natoms
492         do l = 1, 3
493            if (r(l,is) .lt. 0.d0) r(l,is) = r(l,is) + boxlength
494            if (r(l,is) .gt. boxlength) r(l,is) = r(l,is) - boxlength
495         end do
496      end do
497
498      return
```

```
499   end subroutine velpos
```

Listing 17: Full leap-frog program used for the three exercises.